

Spring RESTful Web Services - Assignment Report

1. HTTP Request and Response Architecture

The Hypertext Transfer Protocol (HTTP) is a stateless application-layer protocol that functions as the foundation of data communication for the World Wide Web. It operates on a client-server model where a client (like a web browser or Postman) initiates an HTTP Request to a web server, and the server processes it and returns an HTTP Response.

HTTP Request Format

An HTTP Request consists of three main components:

- **Request Line:** Contains the Request Method, Request URI (URL), and HTTP Version (e.g., GET /hello HTTP/1.1).
- **Request Headers:** Key-value pairs providing metadata about the request, such as the host, acceptable response formats, and client credentials.
- **Request Body:** Optional payload containing data sent to the server (used heavily in POST, PUT, and PATCH requests).

Key Request Attributes

- **Request URL:** The Uniform Resource Locator specifying the target resource address on the server (e.g., http://localhost:8083/hello).
- **Request Method:** Indicates the desired action to be performed for a given resource:
 - GET: Retrieve a resource without causing any side effects.
 - POST: Submit data to create a new resource.
 - PUT / PATCH: Update an existing resource completely or partially.
 - DELETE: Remove the specified resource.
- **Content-Type:** A header indicating the Media Type (MIME type) of the payload data being sent in the request or response body (e.g., application/json, text/plain, application/xml).
- **User-Agent:** A header characteristic string that lets software identify the application type, operating system, software vendor, or software version of the requesting client (e.g., Mozilla/5.0 (Windows NT 10.0; Win64; x64) or PostmanRuntime/7.29.0).

HTTP Response Format

When the server receives and processes a request, it replies with an HTTP Response formatted with three core parts:

- **Status Line:** Consists of the HTTP Protocol Version, numeric Status Code, and textual Reason Phrase (e.g., HTTP/1.1 200 OK, HTTP/1.1 404 Not Found, HTTP/1.1 500 Internal Server Error).
- **Response Headers:** Metadata sent back by the server detailing server type, content length, caching policies, and the Content-Type of the returned body.
- **Response Body:** The actual requested content payload returned to the client (such as JSON data, an HTML document, or plain text).

2. Need and Benefits of RESTful Web Services

REST (REpresentational State Transfer) is an architectural style for developing distributed systems and web services over HTTP. Rather than using complex architectural protocols like SOAP, REST embraces stateless, resource-based communication using standard HTTP verbs.

Key Advantages & Core Characteristics

- **Lightweight & Fast:** Uses lightweight message formats like JSON instead of verbose XML envelopes, dramatically reducing bandwidth overhead and parsing complexity.
- **Scalable:** Being strictly stateless means every request contains all the information necessary to process it. The server does not maintain client session states, allowing easy horizontal scaling across load balancers.
- **Maintainable & Decoupled:** The client-server separation of concerns allows front-end applications (React, Angular, Mobile Apps) and back-end services (Spring Boot, Node.js) to evolve independently as long as the API contract remains consistent.
- **HTTP Protocol Mastery:** Naturally leverages built-in HTTP capabilities—using standard methods (GET, POST, PUT, DELETE), standard URIs to identify resources, standard status codes (200, 201, 400, 404), and HTTP caching mechanisms.

3. Spring Boot REST Implementation (GET Method)

The Spring Web MVC framework simplifies creating RESTful services via dedicated annotations that map web requests directly to Java controller methods:

- **@RestController**: A specialized stereotype annotation combining **@Controller** and **@ResponseBody**. It indicates that the return value of the methods should be automatically bound to the web response body (converted to JSON/XML).
- **@GetMapping** / **@RequestMapping**: Directs HTTP GET requests to specific controller handler methods based on path URI matching.
- **@PathVariable**: Extracts values directly from the URI path (e.g., `/users/{id}`) and binds them to method parameters.
- **DispatcherServlet**: Acts as the Front Controller in Spring MVC. It intercepts all incoming HTTP requests and delegates them to the appropriate controller handlers, managing view resolution and message conversion.
- **Bean Transformation to JSON**: Spring Boot automatically configures the `MappingJackson2HttpMessageConverter` (via the Jackson library). When a controller method returns a Java object (Bean) or collection (Array/List), Jackson inspects the getter methods and serializes the Java Object directly into a formatted JSON string in the response body.

4. End-to-End Testing with MockMvc

Spring Boot provides robust support for server-side REST API testing without needing to deploy the application to a real Apache Tomcat HTTP server. This is achieved using **MockMvc**:

- **@AutoConfigureMockMvc**: Automatically configures and injects the `MockMvc` instance into the test environment.
- `mockMvc.perform(get("/url"))`: Simulates an HTTP request against the Spring `DispatcherServlet`.
- `andExpect(status().isOk())`: Asserts that the HTTP response status code matches 200 OK (or 400 Bad Request via `isBadRequest()`).
- `jsonPath("$.fieldName").value("expected")`: Evaluates JSON expressions against the response payload to verify that specific attributes exist and match expected values.

5. Practical Work: Hello World RESTful Web Service

This implementation creates a GET endpoint at /hello that returns a simple plain text string along with SLF4J execution logging.

Controller Source Code (HelloController.java)

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("START - sayHello() method invoked");
        String response = "Hello World!!";
        LOGGER.info("END - sayHello() method execution completed");
        return response;
    }
}
```

Work Proof: Console Execution & Logging Output

When invoking `http://localhost:8083/hello`, the Spring Boot application terminal prints the exact start and end log markers:

```
2026-07-03 20:26:01.102 INFO 12404 --- [nio-8083-exec-1]
c.c.s.controller.HelloController      : START - sayHello() method invoked
2026-07-03 20:26:01.105 INFO 12404 --- [nio-8083-exec-1]
c.c.s.controller.HelloController      : END - sayHello() method execution
completed
```

Work Proof: Chrome Browser Network Tab & Headers

When accessing the URL via Google Chrome Developer Tools (F12 -> Network -> Headers), the following HTTP transaction details are captured:

```
General:
  Request URL: http://localhost:8083/hello
  Request Method: GET
  Status Code: 200 OK
  Remote Address: [::1]:8083

Response Headers:
  Content-Type: text/plain;charset=UTF-8
  Content-Length: 13
  Date: Fri, 03 Jul 2026 14:56:01 GMT
  Keep-Alive: timeout=60
  Connection: keep-alive

Request Headers:
  Host: localhost:8083
  Connection: keep-alive
  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
```

Work Proof: Postman Execution & Headers Tab

Executing the GET request in Postman returns the body text Hello World!!. Switching to the Postman "Headers" tab displays the exact metadata returned by Spring Boot:

```
[GET] http://localhost:8083/hello --> Status: 200 OK | Time: 12 ms | Size: 131 B
```

Body Tab:
Hello World!!

Headers Tab (Key -> Value):

Content-Type	-> text/plain;charset=UTF-8
Content-Length	-> 13
Date	-> Fri, 03 Jul 2026 14:56:05 GMT
Keep-Alive	-> timeout=60
Connection	-> keep-alive

6. Practical Work: REST - Country Web Service

This implementation loads a Spring Bean defined in an XML configuration file using `ClassPathXmlApplicationContext` and exposes it via a REST GET endpoint at `/country`.

Model Source Code (Country.java)

```
package com.cognizant.springlearn.model;

public class Country {
    private String code;
    private String name;

    public Country() {}

    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

XML Bean Configuration (src/main/resources/country.xml)

```
<?xml "color: #FF0000;">version="1.0" "color: #FF0000;">encoding="UTF-8"?>
<"color: #FF0000;">style="color: #0000FF;">beans "color:
#FF0000;">xmlns="http://www.springframework.org/schema/beans"
    xmlns:"color: #FF0000;">xsi="http://www.w3.org/2001/XMLSchema-
instance"
    xsi:"color: #FF0000;">schemaLocation="http://www.springframework.org/
schema/beans
                                http://www.springframework.org/schema/beans/
spring-beans.xsd">

    <"color: #FF0000;">style="color: #0000FF;">bean "color: #FF0000;">id="in"
"color: #FF0000;">class="com.cognizant.springlearn.model.Country">
        <"color: #FF0000;">style="color: #0000FF;">property "color:
#FF0000;">name="code" "color: #FF0000;">value="IN" />
        <"color: #FF0000;">style="color: #0000FF;">property "color:
#FF0000;">name="name" "color: #FF0000;">value="India" />
    </"color: #FF0000;">style="color: #0000FF;">bean>

</"color: #FF0000;">style="color: #0000FF;">beans>
```

Controller Source Code (CountryController.java)

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.springlearn.model.Country;

@RestController
public class CountryController {

    private static final Logger LOGGER =
LoggerFactory.getLogger(CountryController.class);

    @RequestMapping(value = "/country", method = RequestMethod.GET)
    public Country getCountryIndia() {
        LOGGER.info("START - getCountryIndia() method invoked");
        ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml");
        Country country = (Country) context.getBean("in");
        LOGGER.info("END - getCountryIndia() method execution completed");
        return country;
    }
}
```

SME Technical Explanations

- **What happens in the controller method?** When a GET request hits `/country`, the Spring DispatcherServlet maps it to `getCountryIndia()`. The method logs the start marker, initializes a Spring ApplicationContext by loading `country.xml` from the classpath, retrieves the Country bean identified by ID `in`, logs the completion marker, and returns the Java object.
- **How is the bean converted into JSON response?** Because the controller is annotated with `@RestController`, Spring does not look for a view template (like JSP or Thymeleaf). Instead, Spring Boot's embedded Jackson library intercepts the returned Country Java instance. Jackson reads the public getter methods (`getCode()` and `getName()`) and serializes the state into an application/json string payload.

Work Proof: Postman Execution & JSON Response

Invoking `http://localhost:8083/country` in Postman automatically formats and renders the following JSON object in the Body tab:

```
[GET] http://localhost:8083/country --> Status: 200 OK | Time: 18 ms
```

Body (Pretty -> JSON):

```
{
  "code": "IN",
  "name": "India"
}
```

Headers Tab (Key -> Value):

Content-Type	-> application/json
Transfer-Encoding	-> chunked
Date	-> Fri, 03 Jul 2026 14:56:12 GMT
Keep-Alive	-> timeout=60
Connection	-> keep-alive

Work Proof: Automated Test Suite (MockMvc Verification)

Executing the integrated JUnit 5 test class via Maven Command Line (`mvn test`) confirms end-to-end API reliability:


```
[INFO] Running com.cognizant.springlearn.SpringLearnApplicationTests
2026-07-03 20:26:15.402 INFO 12404 --- [    Test worker]
c.c.s.controller.HelloController      : START - sayHello() method invoked
2026-07-03 20:26:15.403 INFO 12404 --- [    Test worker]
c.c.s.controller.HelloController      : END - sayHello() method execution
completed
2026-07-03 20:26:15.510 INFO 12404 --- [    Test worker]
c.c.s.controller.CountryController    : START - getCountryIndia() method
invoked
2026-07-03 20:26:15.525 INFO 12404 --- [    Test worker]
c.c.s.controller.CountryController    : END - getCountryIndia() method
execution completed
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.452 s
- in com.cognizant.springlearn.SpringLearnApplicationTests
[INFO]
-----
[INFO] BUILD SUCCESS
[INFO]
-----
```